CÓDIGOCLARO · MATERIAL DE ESTUDIO

XPath avanzado y procesamiento XML

Contexto, ejes, XPath 3.1, XQuery 3.1, XSD y XSLT

1. Árbol, nodo de contexto y resultado

Objetivo: Razonar sobre el árbol XPath antes de escribir una expresión que dependa del nodo actual.

XPath evalúa expresiones sobre un modelo de datos con nodos y valores atómicos. El contexto incluye el nodo actual, posición y tamaño de secuencia, junto con bindings de variables y namespaces que aporta el procesador. Una expresión como `.` selecciona el elemento de contexto; `..` su padre y `@id` un atributo. El resultado no tiene por qué ser un nodo: puede ser una secuencia, número, booleano o cadena. La versión del procesador y su API determinan qué funciones están disponibles y cómo se convierten los resultados. Al depurar, registra el nodo de contexto y prueba cada paso por separado antes de anidar filtros.

Código · ejemplo

```
[XPath 3.1]
/c:catalogo/c:libro[price gt 20]/title
```

Práctica: Si el contexto es el segundo `libro`, ¿qué devuelve `../libro` y por qué?

Respuesta razonada: Devuelve los elementos `libro` hijos del padre del nodo actual, no solo el segundo elemento. El paso `..` sube al padre y el paso `libro` vuelve a seleccionar sus hijos con ese nombre y namespace. Si se quería seleccionar el hermano anterior, se debe escoger un eje como `preceding-sibling` y limitarlo explícitamente.

2. Ejes directos e inversos

Objetivo: Elegir el eje XPath que expresa la relación entre nodos sin depender de rutas frágiles.

Los ejes `child`, `parent`, `self`, `descendant` y `ancestor` navegan relaciones estructurales. Los ejes `following-sibling` y `preceding-sibling` encuentran hermanos, mientras que `following` y `preceding` abarcan más nodos del documento con excepciones para descendientes y ancestros. Algunos ejes, como `ancestor`, son inversos: su orden de proximidad importa al aplicar predicados. `//` abrevia una búsqueda de descendientes, pero puede incluir coincidencias no deseadas y costar más. Prefiere expresar una relación concreta y añade condiciones de atributo cuando la identidad de negocio no depende de la posición documental.

Código · ejemplo

```
[XPath 3.1]
preceding-sibling::c:capitulo[1]
```

Práctica: ¿Qué selecciona `[1]` en el eje inverso `preceding-sibling` y qué deberías verificar en tu biblioteca?

Respuesta razonada: En el modelo XPath, el primer resultado del eje inverso es el hermano precedente más próximo, así que la expresión busca ese capítulo. Comprueba que tu procesador evalúe la abreviatura conforme a XPath y que el namespace `c` esté enlazado. Para evitar confusión, compara también la versión expandida y crea una prueba con tres hermanos.

3. Predicados y posición por paso

Objetivo: Controlar el contexto de cada predicado y evitar errores al filtrar y seleccionar posiciones.

La expresión `item[1]` selecciona items que son primeros en el contexto de su paso; `item[position() = last()]` selecciona el último. En una ruta con pasos sucesivos, cada predicado ve una secuencia concreta, por lo que mover corchetes puede cambiar el significado. Los ejes inversos afectan el orden usado para evaluar la posición, mientras que el resultado de una ruta suele quedar en orden documental. No confíes en índices para identificar entidades si se reordena el XML. Escribe conjuntos pequeños de prueba que muestren diferencias: nodo sin hermanos, dos coincidencias, primera que no cumple, y namespaces presentes.

Código · ejemplo

```
[XPath 3.1]
/c:curso/c:alumno[@activo = 'sí'][position() le 3]
```

Práctica: Explica por qué filtrar `@activo` antes de tomar los tres primeros importa.

Respuesta razonada: Primero se forma la secuencia de alumnos activos; `position() le 3` conserva hasta tres de esa secuencia. Si se toma la primera página de alumnos antes de filtrar, podrían aparecer menos de tres activos aunque existan otros después. Para paginación real, añade orden estable por clave, pues posición documental por sí sola puede cambiar.

4. Funciones y conversión de valores

Objetivo: Usar funciones XPath con conversiones explícitas para cadenas, números y valores ausentes.

XPath ofrece funciones para cadenas, números, secuencias y nodos. `normalize-space()` recorta extremos y colapsa secuencias de espacios; `contains()` sirve para subcadenas, pero no equivale a comparación de tokens. Las comparaciones generales `=` y `!=` pueden comparar pares dentro de secuencias, mientras que `eq` exige cardinalidad compatible en XPath 2.0 o posterior. Conversiones numéricas pueden producir errores o valores especiales según la función y el tipo. No compares formatos de fecha como texto si el orden cronológico requiere reglas de zona horaria. Haz explícita la versión de XPath y prueba datos vacíos, inválidos y con espacios.

Código · ejemplo

```
[XPath 3.1]
normalize-space(c:nombre) = 'Ana Ruiz'
```

Práctica: ¿Por qué `contains(normalize-space(.), 'Ana')` no comprueba que el nombre sea exactamente Ana?

Respuesta razonada: `contains` acepta cualquier texto que incluya esa secuencia; también coincide con “Ana Ruiz” y “Mariana”. La normalización arregla espacios, no cambia una búsqueda parcial en igualdad. Para identidad textual normalizada compara `normalize-space(.) = 'Ana'`; si el idioma y mayúsculas importan, define además la política de normalización.

5. Namespaces estáticos en XPath

Objetivo: Resolver prefijos de consulta de forma independiente a los alias usados en el documento XML.

XPath puro es una expresión: el procesador recibe los prefijos en su contexto estático, normalmente configurado mediante la API que evalúa XPath. La sentencia `declare namespace` pertenece al prólogo de XQuery 3.1; por eso el ejemplo completo es XQuery y su cuerpo contiene una expresión XPath. Un prefijo en consulta puede ser diferente al del XML si ambos se vinculan a la misma URI. El namespace predeterminado del documento no siempre se aplica automáticamente a nombres sin prefijo en XPath; esto depende de la API o de una declaración de namespace por defecto para expresiones. Los atributos sin prefijo no heredan el namespace por defecto de elementos. Una consulta sin namespace puede fallar silenciosamente al devolver una secuencia vacía. Registra prefijos y URIs junto a la consulta, y prueba documentos cuyo prefijo físico cambie.

Código · ejemplo

```
[XQuery 3.1]
declare namespace e = "urn:escuela";
//e:alumno[@codigo = "a17"]
```

Práctica: El XML cambia de prefijo `e` a `x`, con la URI intacta. ¿Hay que cambiar la expresión?

Respuesta razonada: No, el prefijo usado por la expresión se enlaza en el contexto estático y no depende del alias escrito en el archivo. Si cambia la URI, sí se debe actualizar el vínculo. Si la API no permite declarar namespaces en la propia expresión, pasa el binding por su mecanismo de contexto y documenta ese detalle.

6. Secuencias y tipos en XPath 3.1

Objetivo: Aprovechar secuencias y tipos de XPath 3.1 sin asumir compatibilidad con procesadores XPath 1.0.

XPath 3.1 permite secuencias de valores atómicos y nodos, y define operadores de comparación por valor y por secuencia. Una secuencia vacía `()` es distinta de una cadena vacía `""` y de `false()`. El operador de concatenación de cadenas `||` y expresiones de tipos son funciones del lenguaje, no equivalentes en todas las versiones antiguas. Un procesador XPath 1.0 rechazará sintaxis moderna o interpretará distinto algunas comparaciones. Define explícitamente el nivel mínimo de soporte del motor y ofrece una alternativa si el consumidor usa una biblioteca antigua. Mantén conversiones explícitas cuando la cardinalidad puede ser cero o más de uno.

Código · ejemplo

```
[XPath 3.1]
let $ids := ("a1", "a2") return string-join($ids, ", ")
```

Práctica: ¿Qué resultado representa `()` frente a `""` y por qué puede importar al consumir la consulta?

Respuesta razonada: `()` representa una secuencia sin elementos, mientras `""` es una cadena existente de longitud cero. Una API puede traducir la primera a “sin resultado” y la segunda a un campo encontrado pero vacío. Si se esperaba un único nodo, declara cómo debe tratarse ausencia, duplicidad y conversión, y verifica que el motor soporte XPath 3.1.

7. FLWOR de XQuery para transformar secuencias

Objetivo: Combinar `for`, `let`, `where`, `order by` y `return` de XQuery 3.1 en una transformación legible.

XQuery 3.1 ofrece expresiones FLWOR completas. En XPath 3.1, `for` y `let` usan cláusulas simples con `return`; `where` y `order by` pertenecen a XQuery. El prólogo enlaza `c` con la URI del vocabulario, y XQuery predefine `xs` para los tipos XML Schema. Una ordenación necesita una clave y regla para valores ausentes o empates. Construir nodos nuevos requiere sintaxis y namespace correctos. Usa variables para explicar cada etapa, pero evita consultas monolíticas difíciles de mantener. Este FLWOR requiere un procesador compatible con XQuery 3.1; anota motor y versión en el proyecto.

Código · ejemplo

```
[XQuery 3.1]
declare namespace c = "urn:escuela";
for $a in /c:curso/c:alumno
let $nota := xs:decimal($a/c:nota)
where $nota ge 7
order by $nota descending
return $a/c:nombre
```

Práctica: ¿Qué ventaja aporta convertir la nota a decimal antes de ordenar?

Respuesta razonada: La conversión hace que el orden sea numérico: de otro modo valores textuales como `10` y `8` pueden ordenarse lexicográficamente en herramientas o transformaciones que traten cadenas. Los datos no válidos pueden causar error al convertir, así que el esquema debería restringir la nota o la consulta debe definir una política de validación y error.

8. XSD: tipos y restricciones

Objetivo: Validar estructura y tipos XML con un esquema XSD que represente restricciones reales del contrato.

XML Schema define elementos, atributos, tipos simples y complejos, cardinalidad y restricciones como enumeraciones o rangos. Un XSD puede validar que el elemento tenga un decimal dentro de límites, pero no expresa cómodamente todas las reglas entre registros o externas al documento. La validación depende de asociar el esquema de forma segura; no descargues automáticamente esquemas indicados por contenido no confiable. XSD 1.1 añade capacidades como aserciones que no todos los validadores implementan. Comprueba la versión del motor y valida tanto documentos correctos como casos límite. Mantén el esquema junto al control de cambios del contrato y comunica incompatibilidades de versión a productores y consumidores.

Código · ejemplo

```
[XSD 1.1]
<xs:element name="nota" type="xs:decimal" minOccurs="1" maxOccurs="1"/>
```

Práctica: Una nota debe estar entre cero y diez. ¿Qué valida este fragmento y qué regla falta?

Respuesta razonada: El fragmento declara un decimal obligatorio y único, pero no establece el rango. Deben añadirse facetas de mínimo y máximo apropiadas al tipo decimal y al nivel XSD elegido. También comprueba que el elemento padre use esa declaración y que la herramienta realmente soporte la versión 1.1 si el esquema usa características añadidas.

9. XSLT y XPath como pareja

Objetivo: Entender cómo XPath selecciona nodos dentro de reglas XSLT y cómo verificar la transformación.

XSLT aplica plantillas a árboles XML; los patrones y expresiones XPath seleccionan nodos y valores. Una plantilla puede procesar una clase de nodo y delegar en sus hijos mediante `apply-templates`, o producir una salida con `value-of`. Las reglas de prioridad y el modo de transformación afectan qué plantilla se ejecuta. En XSLT 3.0, mapas, arrays y funciones amplían el modelo, pero el procesador debe implementarlo. No mezcles transformación con lógica de autorización: una vista de salida no sustituye controles en servidor. Compara la salida con casos de entrada conocidos y conserva trazabilidad entre versión del XSLT y salida desplegada.

Código · ejemplo

```
[XSLT 3.0]
<xsl:template match="c:alumno">
  <li><xsl:value-of select="c:nombre"/></li>
</xsl:template>
```

Práctica: Si ninguna plantilla propia coincide con un nodo, ¿por qué conviene declarar una plantilla raíz?

Respuesta razonada: El procesamiento integrado puede copiar o descartar nodos según las reglas por defecto del procesador. Una plantilla raíz explícita documenta el punto de entrada y la estructura de salida esperada. También permite controlar qué nodos se recorren y evita exponer accidentalmente texto o elementos fuera del resultado previsto.

10. Depuración, compatibilidad y rendimiento

Objetivo: Diseñar pruebas de consultas XPath y evitar errores de versión, namespace y selección demasiado amplia.

Una consulta debe tratarse como código: identifica procesador y versión, conserva ejemplos mínimos de entrada y comprueba los resultados esperados. Si devuelve vacío, revisa primero nodo de contexto, namespace, mayúsculas y estructura real. Si devuelve demasiado, reemplaza `//` indiscriminado por relaciones de ruta más concretas y añade filtros. Ejecutar XPath sobre documentos hostiles requiere límites de tamaño, profundidad y tiempo; una expresión costosa puede afectar la disponibilidad. No interpolar directamente entrada de usuario en una expresión: usa variables enlazadas por la API. Mide sobre documentos representativos y evita optimizar antes de tener una prueba de corrección.

Código · ejemplo

```
[XPath 3.1]
$doc//c:pedido[c:total castable as xs:decimal]
```

Práctica: ¿Qué aporta `castable` y qué problema de seguridad no soluciona?

Respuesta razonada: `castable` permite comprobar si un valor puede convertirse al tipo decimal antes de convertirlo y ayuda a no fallar por texto mal formado. No protege frente a expresiones inyectadas, documentos enormes ni consultas costosas. Usa variables del procesador, limita recursos, valida estructura y prueba rendimiento con datos realistas.

Fuentes oficiales

W3C · XPath 3.1 · W3C · XQuery 3.1 · W3C · XML Namespaces · W3C · XML Schema 1.1 Part 1 · W3C · XSLT 3.0 · W3C · XML 1.0